Design and Analysis of Algorithms

Module 7: Greedy Algorithms

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

Last week

Last time we learned about complexity categories.

One category was particularly important: NP-complete.

NP-complete is full of useful, interesting problems:

- SAT: useful for package managers, AI, among many other things
- TSP: useful for home delivery and machining
- VC: useful for security systems, city management, many other things

Solving these problems is massively useful for many important, real-world things.

The problem is that everything in NP-complete is exponential time!

This week

But sometimes life throws us a bone: just because the general version of those problems is hard, does not mean that every single instance we encounter is hard.

For example, the TSP is hard, but what if our graph is a rectangular grid of tiles with uniform edge weights? Then it's easy: just follow a lawn-mower pattern.

What makes TSP hard is that normally, choosing to visit a city next means we can't visit it from a city later, which might have a shorter path. This means the decision of which city to visit first requires trying it and seeing how it works out, and then potentially revising our choice later.

Greedy algorithms

Greedy algorithms are those in which we can get an optimal result with locally optimal decision making.

It would be like if we could solve TSP by always visiting the shortest distance (which doesn't normally work).

Greedy algorithms arise when there's some aspect of the problem that makes a simple strategy really effective.

Let's consider video games...

NP-complete game problems

The following problems are NP-hard (probably, I haven't proved it or anything):

Deciding the optimal place for districts in Civilization 6. Districts are little buildings you put down that have an effect on neighboring tiles. Going for the highest immediate bonus is not always optimal, because the district you put down blocks other districts.

Deciding the optimal build-order in a well-balanced RTS. Every building you build ends up changing the resources and strategies available, which may affect future decisions.

Determining the optimal chess move. Some moves are better than others, but determining the ideal move currently requires considering all the opponents moves they could make in response.

These are hard problems, but what if the game were not well balanced?

Greedy algorithms for fake NP-complete games

Civ6: Imagine if there were a district that gave the bonuses of every other district. Now you just have to only build that one. There are no longer any tradeoffs.

RTS: Imagine if one building gave the benefits of all the other buildings and cost the same as the cheapest one. Now spamming that building is the optimal strategy.

What if we're playing a fairy-chess puzzle where there's a piece that can move anywhere and cannot be taken? Just only move that piece.

Notice how if one move becomes "overpowered", the search space for optimum strategies becomes very small and predictable.

That's the essence of a greedy algorithm. It's one that solves what seems like a hard problem, by solving a simple problem over and over again.

Questions?

An example problem

Suppose Alice and Bob are playing " Simple War" with $N > 1$ cards.

The cards each have a number on them from 1 to N, with every number being used exactly once.

The cards are randomly partitioned between Alice and Bob.

Alice and Bob play cards at the same time. They can play any card in their hand.

The highest card wins the play. The winning player takes that card and the card they played and add both to their hand. They continue until a player has won all the cards.

What is the optimum strategy? [Let's think about it first.]

If both players use it, can we determine who will win? And can we bound the time it takes?

Simple war

At first, this seems like a combinatorial problem. Each player has a bunch of cards. Which one do they play? Maybe playing a good card now means we can't play it later...

Except if you win, you get your card back plus the opponent's card. So winning means you gave up nothing. And now you are in a stronger position.

And you maximize your chance of winning by playing your best card.

So the ideal strategy is to always play your maximum card.

And the winner will be whoever has card number N. Determining this takes $\Theta(N)$ if we are given a list of cards, or $\Theta(1)$ if we are told the number of cards at the beginning.

Simple war induction

How would we prove this ideal strategy? You guessed it: induction!

But this time, let's do induction over a set of a particular type. Suppose a set of cards is constructed like this:

- $\emptyset$ is a set of cards.
- If H is a set of cards and c is a card, $H \cup \{c\}$ is a set of cards.

This type is inductive because it has a finite number of constructors. We can use induction on this type to prove statements that begin with $\forall H \in \mathrm{Hands}$

We could just do induction over the size of her hand, but it's a little cleaner sometimes to do induction directly on data structures.

Simple war ideal strategy

We have two cases for induction: one for each constructor. Our goal is to prove that for any hand H, playing the largest card is optimal (i.e., that no other strategy is better).

- Suppose that Alice's hand is empty. Then she has lost; every strategy is identical.
- Suppose that for H, the optimum strategy is $\max H$.
  We must show that for some card c, $\max(H \cup \{c\})$ is the optimum strategy
  There are 3 possibilities for c, 2 of which are valid:

  i. $c > \max H$. By the inductive hypothesis, playing $\max H$ was optimal, so playing an even higher card does not lose against any additional hands.

  ii. $c < \max H$. Then $\max(H \cup \{c\}) = \max H$, and the goal follows from IH.

  iii. $c = \max H$. This is impossible because the cards are distinct

Who will win?

If our goal is to determine the winner, then it's whoever has the card numbered N.

How long does it take to do this? If we are given Alice's hand H, we must scan through it until either we find N (meaning she wins), or until we reach the end (she loses).

Therefore, in the worst case, it is $\Theta(|H|)$

In this case, the fact that we defined our algorithm by linear search lets us re-use the big Θ for linear search.

Questions?

A slight modification

Suppose the game has two rule changes:

- Instead of both players playing at the same time, Bob plays first.
- Now, N is the best card...except for 1, which beats N and absolutely nothing else.

So the questions are:

- Is there still a greedy algorithm for Alice that is optimum?
- Can we predict who will win?

We're still assuming both sides play optimally.

[What do you think?]

inspired by this CodeForces problem.

It's still greedy

The key here is that Bob goes first. So Alice can follow this plan:

- If Alice has N and 1, she plays N every hand regardless of what Bob plays.
- If Alice has N and some other card, but not 1, Alice still wins:
  - If Bob plays 1, Alice plays her other card.
  - If Bob plays another card, Alice plays N.
- If Alice has $N-1$ and 1, she still wins:
  - If Bob plays N, Alice plays 1
  - Otherwise Alice plays $N-1$
- Otherwise, Alice loses
  - If Alice only has N Bob will play 1.
  - If Alice does not have N or $N-1$, Bob will play $N-1$ every hand.

The simplest statement of the rule

Alice wins if she has either:

- N and something else: i.e., $N \in A, \wedge |A| > 1$
- $N - 1$ and 1: i.e., $\{N - 1, 1\} \subseteq A$

Complicated rules

Notice that sometimes the greedy strategy can be a little complex.

In this case, the slight change in rules made the ideal strategy require some adjustment.

One way of thinking of greedy algorithms is that they are an "exploit" for a game.

Think back to games you may have played, when there was an overpowered strategy. Often, the developers "patch" the exploit, but then there is a slightly more complicated strategy that starts working. It's kind of like slowly moving the game from polynomial time to NP-complete by gradually removing greedy strategies.

Most good pure-strategy or puzzle games (pure meaning there's no reflex component) seem like they are NP-complete: hard enough to be deep and not have an obvious exploit, but not so hard that the game cannot be solved with good pattern recognition.

Questions?

Another variant

Alice and Bob get bored of playing games that seem like they always have easy solutions. So they try to make it more complicated again.

Bob proposes a rule change:

- Go back to playing simultaneously
- Instead of playing one card at a time, players can play any number in a trick.
- The winner of the trick is the one with the largest sum.
- The winner keeps all the played cards from both players.

If you risk a lot of cards, you're in trouble if you lose. On the other hand if you don't risk enough, you lose the trick.

[What do we think? Any ideas if there even *is* an ideal strategy?]

Yup, it's still greedy

Play every card you have every trick. If both players play optimally, the winner is whoever has the highest sum.

Let's prove it...

Base case: If you have zero cards, you've lost. So any strategy is the same.

If playing all the cards is optimum, consider if you had one more card. Should you play it or hold back?

The key insight: winning a trick makes you strictly better off. If you lose, you are in a strictly worse position. Therefore, if someone can guarantee they win a trick, they should, and they should do it over and over again.

Yup, it's still greedy (2)

- If you hold back, you might still win, in which case it didn't matter, but playing it doesn't hurt either.
- If you hold back you might lose. You either would have lost anyway, or you lost a trick you could have won if you hadn't held back. So playing the extra card is either equal or better for that trick.
- If you don't hold back, you maximize your chances of winning. If you don't win with all your cards, you never could have won that trick, and you will be even weaker. If you do win, you lose nothing.

You might think "but wait, if I hold back, at least I don't lose that card!". However, if your opponent won, they are in a strictly stronger position. So if that card mattered, then you should have played it, and if it didn't, you are still guaranteed to lose now.

Questions?

One more try

Alice and Bob brainstorm to figure out what keeps going wrong in their game designs.

It feels like the fundamental issue is that winning a trick just makes you better. You don't have to give up anything.

Is there some way to make this no longer be true?

[Weigh in: any rule changes we want to consider?]

One more try (2)

One option to improve the game would be to remove the cards played from further play. I.e., you don't get to pick up all the cards you just played and use them again.

The game proceeds in tricks as before. Both players play simultaneously, and play as many cards as they want.

Both players play with identical decks, so there isn't an advantage to having certain cards. Any card can still be played at any time.

The winner of a trick is whoever's sum is higher. If the sums are equal, no one wins.

One more try (3)

You are required to play at least one card, and the game ends when one player finishes playing every card in their deck.

Your final score is the sum of the cards you won and the cards you didn't play. The sum does not include the cards you played.

So now you are not *strictly better* by playing high cards. If your opponent was going to play a 10, your best play would have been to play an 11. Playing all your cards will get the same result if you have > 10 total sum, but then you will not be able to win any further tricks.

Is there an exploit?

First, let's consider this ruleset. Each of the rules I added was because of an exploit:

- If you got to count cards you played for your score, it would go back to being smart to play all your cards. You either win the first trick or you draw. If you win, the game is over and you will have more points. If it's a draw, you would have lost if you hadn't played all the cards.
- If you didn't get to count the cards you won, but only the ones in your hand, your incentive would be to play one card at a time. You either draw or win with this strategy.

Is there an exploit? (2)

- If you could play zero cards, that would then be ideal. The opponent would not get anything for winning tricks. Both players would cross their arms and refuse to play.
- If players did not play simultaneously, the best move would always be to add 1 to the opponents play if available, and your smallest card if you couldn't.

A fragile game

However, I can't *prove* to you that there is no greedy algorithm that would still work here, without "solving" the game.

And the fact that I had to "patch" the game so many times should make us wonder: is there really no way to greed? Why should we be sure this time?

There's no obvious answer. You might say "well, greedy algorithms are always polynomial time, so prove there can't be one for this algorithm". But for NP-complete games, proving that there's no polynomial-time algorithm would be akin to proving P $\neq$ NP.

Game theory

Ideally, we end up with a game in which there is no obvious one move...

Which means that we have to try to guess what our opponent will do...

Which means that any "bias" by our opponent becomes something to exploit. Imagine playing rock-paper-scissors against someone who had a higher than $1/3$ chance of picking rock. You would win in the long run by shifting your picks toward paper.

However, especially when a move that seems good now ends up hurting us later, we have to consider strings of moves. We start to feel the exponential nature of the move space. Think about how Chess is still unsolved despite enormous resources being poured into it.

But game theory is outside the scope of this class. Just remember that when you're looking for a greedy algorithm, you often don't have to use it!

Questions?

One lens to view these card games through

The card games with greedy algorithms actually have a fundamental property. A kind of "greedy essence" that tells you right off the bat that they are greedy if you notice it.

This essential property is the decision we have to make can be encoded as a matroid.

What is a matroid?

Matroid is a scary sounding word, but "-oid" as a suffix just means "like"

So it's something that is like a Matrix.

In what way? In the way that it describes something that is linearly independent.

And really, it is this independence that is kind of a first step that makes greedy strategies a possibility for a problem.

Linear independence

A linear system is one in which the variables don't depend on each other. If we have two variables, taking more of one does not require us to take more (or less) of another.

In the case of many greedy problems, this is the key. If there isn't some drawback to increasing one variable, we can just always do it. If your goal is to maximize x, and x does not affect y or z, then your decision is easy: ignore y and z.

All matrices are matroids, but there are matroids that aren't matrices. There are lots of ways to define a matroid, but there is one that works well for many greedy problems and which doesn't require higher math: the set theoretic definition.

The set definition of a matroid

A matroid is defined by two sets: E and I.

E is the set of **E**lements. These are usually the "things" that you can take or not take; or play or not play.

I is the set of **I**ndependent subsets. Each element of I is a set which consists of zero or more elements of E. These sets represent combinations of choices in the game. I must have the following properties:

- $\emptyset \in I$
- "heredity": $A \in I \implies B \subset A \implies B \in I$
- "exchange": $A, B \in I$ and $|A| > |B|$, then $\exists a \in A, \{a\} \cup B \in I$

Matroids for Alice and Bob's simple card game

Recall that card game where both Alice and Bob could play as many cards as they wanted, and the winner got to keep all the cards they played as well as the cards they won.

Suppose there were 10 cards in Alice's hand. Then $E = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

Then I is the set of all 2^{10} combinations of moves that Alice can make.

E.g., $\{\} \in I, \{1\} \in I, \{2\} \in I, \{1, 2\} \in I, \{1, 2, 5, 7, 10\} \in I$, etc.

Basically, Alice can play any combination of the values of E.

Technically she can't play the empty set according to our rules, but we can treat $\{\}$ as forfeiting the game or something.

Is this a matroid?

We've given E and I. Now we must show that I obeys the heredity and exchange properties:

- Heredity: If $A \subset B$, and $B \subseteq I$ is a valid move, then A is a valid move. Is that true? This is a fancy way of saying, if playing a set of cards is a move, then playing fewer cards is also a move. This is true: the game does not require that we play a certain minimum number of cards.
- Exchange: If $A, B \in I$ and $|A| > |B|$, then $\exists a \in A, \{a\} \cup B \in I$
  Meaning: suppose there are two hands that Alice can play: A and B. If A has more cards, then there's a least one card that we could copy from A and add to B to make it even bigger, and the result from doing that would

So the moves available to Alice form a matroid. Now: what's the best move?

Rado-edmonds

There is a theorem, attributed to Jack Edmonds and Richard Rado, which tells us that optimization problems that can be encoded as matroids have greedy solutions.

In this case, the problem is: "what is the largest value play".

As you would expect, the answer is "the one with the most cards". There's nothing groundbreaking here, but stating it in the language of matroids gives us a way to quickly validate that a greedy strategy is even available.

In this case, we recognize that we can assign a weight to each set of cards Alice can play, and that adding more cards always increases that weight.

Still need induction

Encoding the decision as a matroid helps us know what the "greedy" solution of the decision is.

But it doesn't prove that a greedy algorithm is optimum. We still need to prove that if we follow that greedy algorithm, we end up with an optimum solution.

So the inductive proof would still be required in this case. The matroid is just a way of showing "yes, there is a situation here where we can keep making our move bigger until it involves all the cards. Therefore greed is an option."

If the *whole problem*, including the moves, can be encoded as a matroid, *then* that really is a proof that a greedy algorithm is appropriate.

Questions?

Can we make a matroid?

Alice is learning a new skill. It could be anything: writing, math, Street Fighter 6.

Alice's skill level at whatever this is is $s = 0$, which is a natural number.

She knows that to get better at a skill, you need to challenge yourself with little tests. Therefore, she creates a challenge bank of training opportunities.

Each challenge has two skill ratings: s_{lo} and s_{hi}. Alice can benefit from the challenge if her skill is between $s_{lo} \leq s \leq s_{hi}$. Afterwards, she will gain one point of skill.

If $s < s_{lo}$, the challenge is too hard, and Alice cannot complete it.
If $s_{hi} < s$, the challenge is too easy, and Alice does not gain benefit from it

This was inspired by a PACNW regional problem that I can't find. It involved a student either choosing to study or not based on the skill range of a problem. Please submit a PR if you find it!

Another problem (2)

The input to the program is a list of pairs: $s_{i,lo}$ and $s_{i,hi}$, for each challenge i.

Alice can consider the list in any order. If she is eligable to perform a challenge, she gains one point of skill, but now may be ineligible to perform some challenges later.

Alice is permitted to do a challenge she is ineligible for, it just doesn't give her a skill point.

The goal is to compute the set of challenges that Maximize Alice's skill points gained.

Another problem (3)

Here's an example, suppose this is an input:

```
0 10; 1 10; 2 10
```

In this case, there are three challenges with ranges 0 to 10, 1 to 10, and 2 to 10. Alice should take all three. She ends up with 3 points of skill. She does not benefit from skipping any.

Here's another example input:

```
0 10; 0 0; 1 1
```

Here, if she takes the first challenge, the second doesn't help. But if she skips the first challenge, the second helps. So the answer is 2 points. One from $\{0, 2\}$, but there are others.

Another problem (4)

So the question is: how do we do this? What is an algorithm that is both optimal (we get the maximum number of skill points) while also being as fast as possible.

If Alice takes a challenge, it makes other challenges less valuable, which seems to imply that there is some thinking we need to do, but maybe not...

Is this NP-complete? Harder? NP-inter? P?

[what do you think?]

This is greedy and P time

Alice should take every challenge. She should not skip a challenge. The optimum result is just be the set of all challenges.

First, if you agree, try to formulate why.

If you disagree, try to find a counter example.

Building a matroid

Here, our problem forms a matroid.

I is the set of decisions we could make, so let's let I be the set of all answers (i.e., the combinations of challenges we could take).

If I is the set of all combinations of challenges themselves, then E must be the set of challenges (i.e, $E = \{0, 1, 2, 3, 4, 5, \ldots\}$)

What are the other requirements?

Building a matroid (2)

Is $\emptyset \in I$? Yes, Alice can just not do any challenges. It's a bad option, but it's an option.

If $B \in I$ and $A \subset B$, then $A \in I$? Yes, Alice can always take one fewer challenge and the result is still valid.

If $|A| > |B|$, then we can copy an element from A in to B to make B', and B' will be in I? Yes, any time there is a problem in set A that is not in set B, we can always add that problem to set B.

So this is a Matroid. And whatever value there is to an answer I, we will never make it worse by adding another problem.

But that's not 100% obvious, so let's prove it.

Inductive proof

Let's do a proof over a list. A list of some type T has two constructors:

- `[]` is a list.
- If `t` is a list, `h : t` is a list, where `h` is an element of type T, and `:` means "cons" (i.e., create a new list with the given element as its head).

Suppose we have a proposition that starts with "for all lists of T". We can prove it by showing that the proposition is true of the empty list, and that if the proposition is true of some list, it's still true if we add a random element to the front of the list.

In this case, our list is a list of pairs: $S_{i,lo}$ and $S_{i,hi}$

Inductive proof (2)

Our goal is to show that always taking a challenge is optimal. That means that another strategy isn't strictly better (although it could be equivalent).

Now, let's represent the amount of skill points Alice gets as a recurrence relation:

- $f(l)$ is the number of points Alice gets from a list of problems l
- $f([]) = 0$
- $f(h : l) =$
  - $f(l) + 1$ if $h_{lo} \leq f(l) \leq h_{hi}$
  - $f(l)$ otherwise

Inductive proof (3)

For the base case: if the list is empty, any strategy is optimal. $f(0) = 0$ for any strat.

For the inductive case: suppose that always taking a challenge is optimal. Now we have a new challenge, $h = (h_{lo}, h_{hi})$. How can we show that we should take it?

- If we take it, either $f(h : l) = f(l)$ or $f(h : l) = f(l) + 1$. That is, either we get a skill point or we don't.
- If we *don't* take it, then we get $f(l)$.
- Therefore, our outcomes are either identical or better if we always take it.

Inductive proof (4)

So, in this case, we were able to fit a matroid to our problem, which is one way of showing that there might be a greedy solution available.

We realized that taking more challenges would result in a better score, but we decided not to stop there, and ended up proving the result inductively anyway.

Ultimately, greedy algorithms generally require inductive proofs. Unless the matroid structure is super obvious, and even if it is, it's worth while to prove it.

Proofs of greedy algorithms typically involve showing that if we deviate from the greedy strategy, we don't end up being better off.

Questions?

Modifications: Alice doesn't waste time

Suppose that Alice is not permitted to waste time. So if she has an input like this: `0 0; 0 0; 1 1`, valid answers are $\{0, 2\}$ and $\{1, 2\}$. However, $\{0, 1, 2\}$ is not valid, because doing problem 1 after problem 0 would be a waste of time (it would not make Alice gain a skill point).

Does this change anything?

Mod 1 (2)

Yes, it means we can no longer encode the whole problem as a matroid.

We actually violate both constraints on I. Heredity isn't there anymore: just because we have a valid member of I, does not mean that we can remove elements of it and have it still be a valid member of I.

Example for the previous slide's example: just because $\{0, 2\}$ is a valid solution, does not mean that $\{2\}$ is a valid solution. In fact, it's not, because Alice's skill starts at 0, and problem 2 requires a skill of 1.

Does that mean that there is no greedy solution?

No, just because we cannot encode the problem as a matroid does not mean that there is no greedy solution.

In this case, there *is* a greedy solution:
Alice should do as many problems as she can at any point. She should keep adding problems to the set until her skill is outside the range of all of them.

There is a broader category of "-oids" here that we can use: a Greedoid

All matroids are greedoids, but greedoids are a little more flexible, and fit more problems.

Greedoids

A greedoid is just like a matroid, in that it can be defined by two sets.

Here, instead of (E, I), we say (E, F), where F stands for "feasible set".

It's still a set of "valid moves", we just call it something slightly different, because greedoids aren't just about linear independence, they're about feasability.

With greedoids, we still have the exchange requirement: if $A, B \in F$ and $|A| > |B|$, there exists some element of A that can be copied into B.

Greedoids (2)

We modify the heredity requirement. Originally, we had to be able to remove *any* element of a solution, and have the result still be a valid solution (i.e., if $A \in I$, then any subset of A is $\in I$).

Here, there's a weaker requirement, called "accessibility":
If $A \in F$, then there exists an $x \in A$ such that $A \setminus \{x\} \in F$
Here: "$\setminus$" means "without". It's the set equivalent of a minus sign.

This is saying: we must be able to remove an element of a valid solution. But not *all* elements, just some element.

Let's show that our problem here can be represented as a greedoid. If we can, then we can assume there is a greedy solution.

Proving accessibility

The goal here is to show that given any non-empty solution, we can always remove one more challenge.

Which challenge can we remove? The one with the highest skill requirement.

If there are 10 challenges in our answer, and the answer is valid, then Alice will have a skill of 10.

That means, every challenge must have a minimum skill requirement ≤ 9, otherwise at least one challenge would have been a waste of time.

There might be a challenge with a requirement of 9. We had to build up to it by doing all the other challenges. After doing it, the skill level is 10. Pick this challenge to remove.

Proving accessibility (2)

Let's build a relation R that represents feasible solutions.

This relation has two ways of constructing elements:

- $\emptyset \in R$, because Alice can do nothing if she wants.
- If $f \in R$ and $a_{lo} \leq |f| \leq a_{hi}$, then $f \cup \{a\} \in R$.

This kind of construction is very common for proving things about algorithms.

This relation has a pair of constructors, just like natural numbers and just like lists. Therefore, we can do induction on it the exact same.

Proving accessibility (3)

We need to show that R is complete. That means that every feasible set $F \in R$.

We need to show that R is correct. That means that every element of R is feasible.

Why? Because we need R to include exactly the same elements as F. If it doesn't, then just because we prove something is true for R does not mean we proved it for F.

Then, we will use R to show the accessibility property and the exchange property.

Proving completeness

Goal: for every $f \in F$, $f \in R$.

If f is empty, then $f \in R$ by definition.

Otherwise, there is some order that we can do the challenges in f and not violate any of the constraints. Call this order $f_1, f_2, \cdots, f_n$

By induction on i:

- $f_0 = \emptyset \in R$ by definition of R.
- If $f = f_1$ up to $f_i \in F \implies f \in R$, then f_1 up to $f_{i+1} \in R$
  Recall that our order was feasible. Therefore, $f_{i+1,lo} \leq i \leq f_{i+1,hi}$, so $f \cup \{f_{i+1}\} \in R$

Proving correctness

Goal: for every $f \in R$, $f \in F$

Now we do induction on "derivations" of $f \in R$.
A derivation is just an element of the relation that we generate from an earlier one.

This may seem strange, but $f \in R$ is actually an inductive claim. We're saying "either f is empty, or there's some smaller solution that we made bigger by adding a valid challenge to it"

So in the same way that we did induction on natural numbers, and induction on lists, we can do induction on derivations of this relation.

If a proposition about the relation is true when $f = \emptyset$, and if it's true for some f where $a_{lo} \leq |f| \leq a_{hi}$, then it's true for the result, $f \cup \{a\}$, then we can say the proposition is true for every member of the relation.

Proving correctness (2)

By induction on derivations of $f \in R$:

- When $f = \emptyset$, it's in F because the empty set is a legal play for Alice.
- If it's true that $f \in R$, and there is some challenge a where $a_{lo} \leq |f| \leq a_{hi}$, then it's true that $f \cup \{a\} \in F$.
  Here, we can assume that there is some legal order to perform the tasks in f. After doing so, Alice's skill level will be $|f|$. Since we know that $|f|$ is between the skill requirements of a, we know that a can be performed after doing everything in f.

So we can see that the moves in R are the same as the moves in F. R is just a way of making it explicit how we compute a move in F.

Proving accessibility (4)

To recall, we want to show that if $f \in F$ and $f \neq \emptyset$, then there is an $a \in f$ such that $f \setminus a \in F$.

Let's do induction on derivations of $f \in F$ again.

- If $f = \emptyset$, this contradicts the hypothesis that $f \neq \emptyset$, so this is vacuously true.
- If The proposition is true for f, and there is an a where $a_{lo} \leq f \leq a_{hi}$, then the proposition is true for $f \cup \{a\}$.
  Here, we choose a to be the element we remove. We obtain f, which we supposed was in F.

The purpose of the relation was to make it explicit how we build-up a feasible move from simpler moves. Accessibility just requires us to be able to make the move smaller. So we use what we know about R to remove the most recent addition.

Proving the exchange property

Recall that there was one more property we needed to show.

If $A, B \in F$, and $|A| > |B|$, then there exists an $x \in A \setminus B$ such that $B \cup \{x\} \in F$

By our completeness property earlier, we know that $A \in F$ implies $A \in R$. We will show that for all $f, f' \in R$, if $|f| > |f'|$, then $\exists a \in f, a \cup f' \in R$

Let's again do induction on $f \in A$:

- If $f = \emptyset$, then it's impossible for $|f| > |f'|$ for any f', because that would require $|f| > 0$, but $|f| = 0$. So this is vacuously true.
- For our inductive case, consider that there is some element $a \in f \setminus f'$ we can copy from f into f'. Is that also true for $f \cup \{a\}$ if $|f \cup \{a\}| > |f'|$? This requires some case matching...

Proving the exchange property (2)

If $|f \cup \{a\}| > |f'|$, there are two possibilities:

1. $|f| > |f'|$. In this case, our goal follows from the inductive hypothesis. We already know there's an element from f that can be copied into f'.
2. $|f| = |f'|$. This is the trickier case. Now we need to actually *find* the element to copy, because we can't use the inductive hypothesis (it required that $|f| > |f'|$) Luckily, we can still use a, becuase the requirement for a was that $a_{lo} \leq |f| \leq a_{hi}$. But here, $|f| = |f'|$, so $a_{lo} \leq |f| \leq a_{hi}$. Because of the derivations of our relation R, $f' \in R \implies a_{lo} \leq |f'| \leq a_{hi} \implies f' \cup \{a\} \in R$. And by the correctness property, $f' \cup \{a\} \in R \implies f' \cup \{a\} \in F$.

So finally, we know that our entire problem meets the requirements for a greedoid, meaning that there is greedy solution.

Reassurance

I know it is overwhelming to have two new induction techniques introduced in the same lecture.

I'm just showing you how there are lots of ways to prove that a greedy algorithm is optimal. We can frame it as a matroid, or a greedoid. We can use induction on natural numbers, or lists, and now, even relations.

When I ask you about greedy algorithms, I will leave it up to you to determine how you want to prove optimality.

Questions?

Practice problems:

1. Consider what happens if some challenges lower Alice's skill. Is that problem still greedy? How would we solve it?
2. Consider what happens if Alice does not get to choose the order of the problems, but instead is given a "take it or leave" it choice for each problem. Is it still greedy?
3. Consider if each problem has an "effort" rating that requires how much work it takes to do, but also a "reward" rating that determines how much skill Alice gets. Alice has 100 effort points to spend. Is this greedy?

Partial answers:

1. This problem is still greedy, but we only want to take challenges that don't lower Alice's skill.
2. This problem is also greedy. Alice should always take a challenge for which she is eligable.
3. This problem is *not* greedy. It's actually an instance of the knapsack problem, which we will learn how to solve next module. In short: if we always take, e.g., the most skill gain, or the highest ratio of skill gain to cost, or something like that, each of those strategies is falsifiable.

Questions?

The quiz

Next module, we'll have a quiz. The quiz will have this format:

1. Here is a problem. Implement a greedy solution in C which is optimal.

That's it.

Why did we have to learn proofs? Because in real life, it won't be obvious that a problem is greedy, and understanding the math will help you recognize it.

However, as you've seen in this section, the proofs can get pretty long, and I don't think it's fair to ask for one on the test.

(Originally I had them on there, but when I was practicing the quizzes myself, I was realizing that it often took me longer than 15 minutes!)

Quiz 1

Solve the following problem in C with a greedy P-time algorithm:
You are given an array of integers `int* arr`, a `size_t n` and a `size_t k`.
Find the maximum sum that can be obtained by choosing `k` integers from `arr`.
Bounds: `0 < k <= n`

Example: `choose_k({1, 2, 3}, 3, 2) == 5`, because the largest sum of 2 integers in the array `{1, 2, 3}` is `2 + 3 == 5`

Quiz 2

Solve the fractional-backpack problem in C with a greedy P-time algo:
You are given an array of this struct:

```
typedef struct item_t { float weight; float value; } Item;
```

You are given a maximum weight. Your goal is to return the maximum value you can carry. You are allowed to carry fractions of items.

E.g., `fract_bp({ {20.0f, 100.0f}, {10.0f, 5.0f}}, 2, 15.0f) == 75.0f`, because we will take 15 of the first item. If 20 is worth 100, then 15 is worth 75.

Quiz 3

You are given a list of jobs, each of which is defined entirely by its deadline `d`, which is the number of days in the future the job is due. You can do one job per day, and every job pays $200 if done on or before the deadline, and $0 otherwise. Write a greedy, P-time C program giving the maximum amount of money you can make in `dt` days.

```
size_t max_money(size_t* jobs, size_t n, size_t dt) { ... }
```

Example: suppose the list is `{0, 1, 5, 2, 1}`. The best we can achieve is doing the day 0 job on day 0, one of the day 1 deadline jobs on day 1, and then the day 2 and 5 jobs on days 2 and 3. That's 4 jobs for $800. It's impossible to do both jobs with a day 1 deadline.

More practice

Go to CodeForces.com. On the left, there's a little filter for problem difficulty. You want 1000 elo difficulty problems.

Almost all of these have greedy solutions. They are often thoughtfully defined problems, and the writeups usually include a proof of greedy optimality.

LeetCode also has greedy problem lists. I recommend doing "Easy" difficulty problems: the ones I quiz you on will not be too challenging.

Grinding is great

Being able to quickly see that a problem is greedy isn't a guaranteed skill. It requires some amount of grinding to see the common patterns.

If you weren't able to get these practice problems within 10 minutes each, I recommend doing some problems on the above sites.

Be sure your solution is accepted! It's tempting to say "I could do that", but the problem writers think of lots of clever corner cases that might not be obvious. It's a good way to test your proof for fallacies.

More encouragement

Greedy algorithms are extremely common in coding interviews and competitive coding competitions. They give the appearance of being challenging combinatorial problems, but they often have short, quick solutions.

They test both programming skills and mathematical reasoning. Usually an inductive proof is at the heart of them.

I think if you get good at these skills (both finding the greedy solution and proving it is optimal), you will be happy you did so.

It's totally normal to feel lost or experience programmer's-block. Go ahead and give yourself a quiz-length time limit to solve a problem, and if you don't get it, peek at a solution or ask an AI. Over time, the proportion you get will go up, and the time taken will go down. The difficulty level on CodeForces is rather high, too, so don't feel bad.

Questions?